

架构基础概念与方法

Architecture Fundamentals: Concepts and Methods

文档版本：v2.11 创建日期：2026-08-11 修订日期：2026-08-11 历史：

- 2026-08-11 v1.0→v2.11 从初版经历20余次迭代，形成七章+两附录完整结构：基本术语与设计链缩写、五条基本原理、架构五层面目的与意义（含系统理论+电商双理论案例）、三层不可逆性判断框架与决策记录六问格式、从相关方推导七候选视图（含用户操作视图+运维者/鉴证者/采办者辨析+物料供应方条件性纳入）、4A模型×七视图交叉矩阵、设计链四层决策全景（STR/BA/SR/PA末级节点对齐91规范v5.8：场景业务→输出文档→系统功能点→产品构件）

一、基本术语

本章定义全文使用的核心术语。定义不引入任何方法论假设——只从概念本身出发，说清楚「是什么」。

1.1 系统 (System)

系统 = 一组相互关联、相互作用的元素，为达成某个共同目的而构成的整体。

（综合自 ISO 9000:2015, §3.5.1 [interrelated or interacting elements]、ISO/IEC/IEEE 15288:2015, §4.1.46 [organized to achieve stated purposes]，及一般系统论 [Bertalanffy, 1968; Ackoff, 1971] 中关于整体性与目的性的论述。四要素框架——元素/关系/边界/目的——为本文档基于上述来源提炼的分析框架。）

一个东西要被称为「系统」，四个要素缺一不可：

要素	含义	缺失的后果
元素	构成系统的组成部分——人、软件、硬件、流程、数据.....	没有元素就没有载体
关系	元素之间的相互作用——数据流、控制流、依赖、协作协议	没有关系，元素只是堆在一起的零件
边界	系统与外界的区分——什么在系统内，什么在外部环境	没有边界就不知道设计范围
目的	系统存在的理由——它要解决什么问题、创造什么价值	没有目的，系统就是无意义的空转

三个重要推论：

a) 系统是相对的。 一个系统的元素本身可以是另一个子系统 (subsystem)，系统又存在于更大的超系统 (supersystem) 之中。系统的这种层级嵌套属性，决定了架构也需要分层描述。

b) 企业本身就是一个系统。 企业里的人、流程、信息、技术工具、规章制度都是元素，它们通过协作关系（审批流、汇报链、订单履约链.....）连接在一起，以「创造和交付价值」为共同目的。

c) 系统的核心挑战不是「有没有元素」，而是「元素之间的关系是否有序」。 一堆无序元素不是系统，是混沌。架构要解决的核心问题正是关系的秩序。

1.2 架构 (Architecture)

架构 = 系统在其环境中的基本概念或属性，体现于其元素、关系以及设计和演化的原则之中。
(ISO/IEC/IEEE 42010:2011, 3.2)

拆解这个定义的关键词：

「基本的」 (fundamental)：架构不关心一切细节，只关心那些如果改变就会让系统变成另一种系统的东西。这划清了架构和详细设计的边界——**架构决策**：修改成本极高、影响范围大、不可逆或几乎不可逆（如选择微服务还是单体、数据所有权按什么域划分）；**详细设计决策**：可在局部范围内修改、不影响系统整体性质（如某个接口用 REST 还是 gRPC、某张表的索引怎么建）。

「元素和关系」：架构的两个基本构件。元素 = 系统的组成部分（构件、模块、服务）；关系 = 元素之间的连接（接口、依赖、数据流、调用拓扑）。一个常见的误区是只关注元素而忽略关系——但系统的涌现属性（emergent properties）往往来自**关系**，而非单个元素。可靠性的瓶颈通常不在单个组件，而在组件之间的交互模式（超时重试风暴、级联故障）。

「设计和演化的原则」：架构不仅是静态的快照，还包含原则——设计原则约束架构决策如何做出（如「数据所有权按业务域划分」），演化原则约束系统如何随时间变化（哪些可以变、哪些不能变、变更按什么流程走）。这意味着架构不是一张图，而是一个**学习系统**——架构知识随时间和经验持续积累：每次架构层面的变更失误都应沉淀为架构原则的更新。

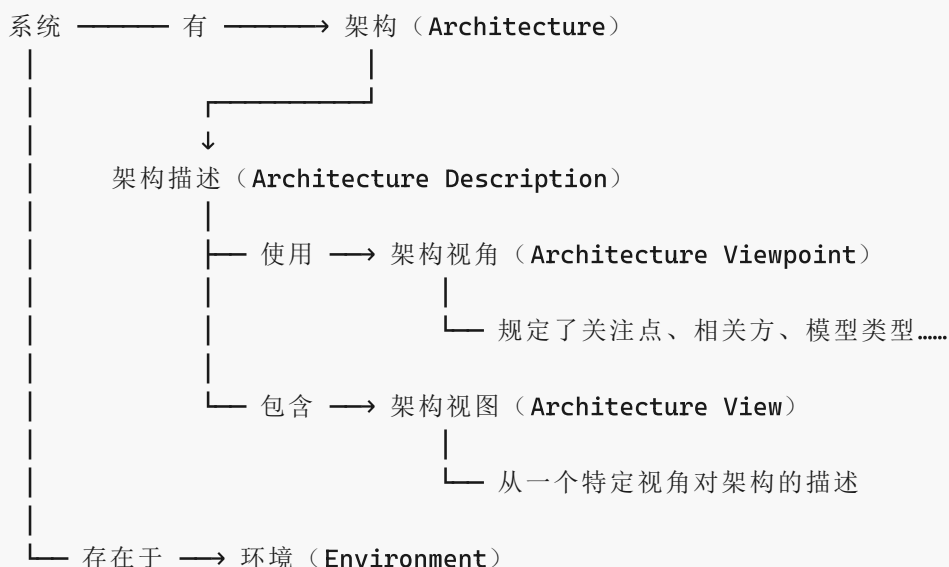
1.3 架构描述、架构视角与架构视图

这是消除概念混淆最关键的一组区分。

一个系统只有一个架构。 那些「业务架构」「应用架构」「数据架构」「技术架构」「需求架构」「功能架构」「逻辑架构」「物理架构」.....不是多个架构，而是对同一个架构的不同描述——从不同视角看、在不同抽象层次上描述。就像一个人只有一副骨骼，但医生可以从正面拍 X 光、从侧面拍 X 光、从不同层面拍 CT 切片——这些都是同一副骨骼的影像，不是多副骨骼。

混乱的根源在于：**日常用语把「架构描述」简称为「架构」，把「架构视图」也简称为「架构」，久而久之，听起来就好像有多个架构了。**

ISO/IEC/IEEE 42010:2011 的架构描述框架 (Architecture Description Framework) 给出了精确的术语体系：



概念	是什么	数量
架构 (Architecture)	系统自身的本质属性——它的元素、关系、和原则	一个系统只有一个
架构描述 (Architecture Description)	用来表达架构的文档/制品集合	一个系统可以有一个架构描述，也可以没有（没有文档不意味着没有架构）
架构视角 (Architecture Viewpoint)	一个观察角度——「我从什么立场看这个架构」，规定了关注点、相关方、模型类型	可以定义多个
架构视图 (Architecture View)	从某个视角出发产出的架构描述的一部分	每个视角产生一个视图

为什么日常听到这么多「XX 架构」——两套分类维度的叠加：

市面上各种「XX 架构」实际上来自**两套独立的分类维度**，叠加在一起就产生了混乱。

维度 1：关注域（按「看什么」切分）——TOGAF 等企业架构框架的分类方式：

传统称呼	实际含义	关注的问题
业务架构	从 业务视角 描述架构	业务能力是什么？业务流程是什么？组织怎么分配？
数据架构	从 数据视角 描述架构	数据资产有哪些？数据从哪来、归谁管、怎么用？
应用架构	从 应用视角 描述架构	有哪些应用系统？它们之间怎么交互？
技术架构	从 技术视角 描述架构	用什么技术平台？基础设施怎么部署？

维度 2：抽象层次（按「多详细」切分）——系统工程传统中的分层方式：

传统称呼	实际含义	关注的问题
需求架构 / 概念架构	最抽象的架构描述	系统要做什么？有哪些相关方？
功能架构 / 逻辑架构	中间的架构描述	系统提供什么功能？逻辑数据流是什么？
物理架构 / 部署架构	最具体的架构描述	功能跑在哪些物理节点上？用什么技术实现？

这两个维度存在**重叠和交叉**。任何一个具体的架构视图都同时位于两个维度的某个交叉点上：

关注域 \ 抽象层次	概念层	逻辑层	物理层
业务	业务能力图	业务流程图	业务组织图
数据	概念数据模型	逻辑数据模型	物理数据模型
应用	应用功能列表	应用交互图	应用部署图
技术	技术选型原则	技术平台架构	网络拓扑图

第三类混淆——架构风格/架构模式：

「微服务架构」「事件驱动架构」「分层架构」「六边形架构」.....这些**不是架构，是架构模式/架构风格——即架构决策的套路**。相当于建筑中的框架结构、剪力墙结构、钢结构——这些是结构体系的选择，不是建筑本身。

三种说法的辨析总结：

说法	实际含义	类比
「业务架构 + 应用架构 + 数据架构 + 技术架构」	同一个架构的四个关注域视角（按管理归属切分）	一栋楼的平面图 + 结构图 + 管线图 + 立面图
「需求架构 → 功能架构 → 逻辑架构 → 物理架构」	同一个架构从抽象到具体的四个描述层次	一栋楼的概念草图 → 方案设计 → 扩初设计 → 施工图
「微服务架构 + 事件驱动架构 + 分层架构.....」	架构模式/架构风格——架构决策的套路	框架结构、剪力墙结构、钢结构——结构体系选择

1.4 架构决策 (Architecture Decision)

架构决策 = 你在设计系统时做出的某个选择，这个选择选错了的话，后来要花很大代价才能改对。

反过来，如果你做了一个选择，后来发现错了，改一下只要几分钟——这不叫架构决策，这就是普通的实现选择。

三个让人立刻有感觉的例子：

例子 1：书架怎么分类。 1000 本书，按颜色、按作者首字母、还是按主题？选了按颜色——以后每次找「Python 相关的书」都要把所有颜色层翻一遍。选了按主题——「计算机」那一层秒找到。选了之后再想从按颜色改成按主题，500 本书全部重新摆——这就是修改成本。「书架按什么规则分类」就是一个架构决策。

例子 2：厨房的布局。 商用厨房里，灶台和冰箱之间的距离。灶台紧挨冰箱 = 效率高但散热互相影响；灶台和冰箱在对角线 = 各自独立但每次拿食材走 8 步。不管选哪个，一旦灶台接好燃气、冰箱接好水电，再想挪位置就得拆墙重铺管线停工几天——这就是不可逆。

例子 3：软件系统中订单服务的边界。 「库存扣减」放在订单服务里还是库存服务里？放在订单服务 = 前期开发快，但三年后采购入库、盘点调整都要改订单服务，订单服务开始管库存的事，职责膨胀。放在库存服务 = 多一次远程调用，但库存的所有操作（扣减、入库、盘点、调拨）都在一个地方，职责干净。「库存扣减放在哪个服务里」就是一个架构决策。

架构决策有三个标志：

标志	含义	反面（非架构决策）
影响半径大	改了它，不止改一处，要联动改很多东西	改它只影响一个文件、一个函数、一个模块内部
持续时间长	一旦选定，后续无数个其他决策都建立在这个决策之上	今天做了，下周就可以换掉，不影响其他东西
修改成本高	选错了要改对，代价大到你会选择「算了，就忍着吧」	选错了，改对只需要一次 commit

用这三个标志套一下常见决策：

决策	影响半径	持续时间	修改成本	是不是架构决策？
系统拆几个服务，每个服务管什么	大（所有后续开发都在这个框里做）	系统整个生命周期	极高（数据迁移+团队重组+接口重定）	✅ 是
订单服务的数据库选 MySQL 还是 PostgreSQL	中（影响一个服务的 schema 设计+运维）	系统生命周期（迁移成本高）	中-高	✅ 是
订单服务的内部代码用哪个 ORM 框架	小（只影响订单服务内部实现）	中（换框架要重写 DAO 层但接口不变）	中（机械劳动，风险可控）	⚠️ 边界
订单列表的排序逻辑用冒泡还是快排	极小（只影响一个查询的性能）	短（随时可以优化）	极低（换算法一次 commit）	❌ 不是

缩写	全称	一句话
STR-F	相关方需求—功能需求	STR 的功能需求分支，按用户角色 × 触发组织
STR-NFR	相关方需求—非功能需求	STR 的非功能需求分支，按角色 → 类目 → 分类组织
BA	业务架构 (Business Architecture)	将功能需求转译为输出文档——「症状拆成可诊断的单元」
SR	系统需求 (System Requirements)	将输出文档转化为系统功能点和技术约束——「确诊并开检查单」
SR-F	系统需求—功能需求	SR 的功能需求分支
SR-NFR	系统需求—非功能需求	SR 的非功能需求分支，从按角色切换为按分类组织
PA	产品架构 (Product Architecture)	将系统功能归并到可实现的产品构件——「开药方」

这些缩写指代的是**设计链中的某一层产出**，不是某一个具体文档。完整的设计链为：**OR → STR → BA → SR → PA**，每一层都有自己独特的架构决策类型（详见第七章）。

二、基本原理

本章从基本术语出发，建立关于架构的几条基本原理。这些原理回答「为什么架构重要」「架构在做什么」「怎么判断架构做对了」。

2.1 一个系统，一个架构

原理陈述：一个系统只有一个架构。市面上所有的「XX 架构」都是对同一个架构从不同视角、在不同抽象层次上的描述（即架构视图），不是多个独立的架构。详见 §1.3 的完整辨析。

为什么这条原理重要：如果以为系统有多个架构，设计工作就会变成「每个架构画一张图，各画各的」——结果是一个系统有多套互不连通的描述，重要的东西可能在所有「架构」的缝隙中漏掉。正确的认知是：**你只有一套架构，你需要的是从不同角度把它描述清楚。**

2.2 架构决策是不可避免的

原理陈述：架构决策不是「做不做」的问题，而是「什么时候做」的问题。你只有两个选择——在成本低的时候主动做，或在成本高的时候被动做。

用一个思想实验来说明。假设你要盖一栋 30 层的办公楼：

- **做法 A：**找几个施工队直接开工。地基挖到一半发现下面是软土层，得加深——返工。墙砌到 5 层发现电梯井位置和管线打架——拆墙重来。封顶后发现消防通道宽度不达标——改不了。

- **做法 B**：先出结构图、管线图、消防图，各专业对图协调，确认无误后按图施工。

做法 A 并不是「没有架构」——它只是把架构决策**推迟到了施工过程中**。地基要多深、承重墙在哪、管线怎么走、电梯井位置在哪——这些决策一个也跑不掉，终究要做。做法 A 的问题不是没有架构，而是**架构决策发生得太晚**——晚到修改成本已经爆炸。

映射回软件/企业系统：

盖楼	做系统
地基	数据模型——改一张核心表的字段类型，成本可能波及几十个服务
承重结构	服务边界——拆分或合并服务，比改内部逻辑贵一个数量级
管线	接口契约——改了上游接口，所有下游全得适配
消防规范	安全合规——事后改造安全模型，往往意味着架构级推翻重做

修改成本曲线说明了这一点：



为什么这条原理重要：它把架构从「要不要」变成了「怎么做好」——消除了「我们不需要架构」这个伪选项。不画架构图不等于没有架构决策。数据存在哪个库、功能拆成几个服务、接口怎么定义——这些决策在写第一行代码时就在发生，只是做这些决策的人可能没意识到它们的不可逆性。

2.3 约束即解放

原理陈述：架构约束不是限制，而是**回收积极自由**。通过消除低价值的决策空间，架构把注意力集中到真正需要创造力的地方。

区分两种自由：

- **消极自由** (freedom from)：不受约束——「我想怎么做都行」
- **积极自由** (freedom to)：有能力高效完成——「我知道怎么做是对的」

没有架构的时候你拥有消极自由（可以随便加服务、随便定义接口、随便放数据），但你丧失了积极自由——每个决策都需要重新推演上下文，永远在「要不要拆服务」「数据放哪个库」「接口怎么定义」这类问题上消耗精力。

架构通过约束来回收积极自由：

架构决策前：

"这个功能的数据应该放哪个库？" → 需要评估 7 个数据库 × 3 种访问模式 = 21 种可能

架构决策后（"订单域的数据全部归属订单服务自有数据库"）：

"这个功能的数据应该放哪个库？" → 订单服务自有库。下一个问题。

为什么这条原理重要：它解释了为什么好的架构不是「管得最多」，而是「管得最准」——只约束那些不改会造成灾难的东西，剩下的留给执行者发挥。架构师的核心能力不是「做出正确的决策」，而是「识别哪些决策如果不正确后果很严重」——对不重要的事情不决策，本身就是一种好的架构决策。

2.4 视图完备性由相关方决定

原理陈述：架构需要哪些视图，不由方法论规定，而由相关方及其关注点决定。视图的数量和内容，取决于哪些人需要理解系统、每个人关心什么。

推导路径：

1. 识别重要相关方 → 2. 明确每个相关方的核心关注点 → 3. 按语言和表达方式聚类关注点 → 4. 每

没有标准数字。3 个也好、7 个也好、12 个也好——只要你知道每一个视图存在的理由（「这个视图在回答谁的什么问题」），且没有重要相关方的核心关注点被遗漏，就是完备的。

为什么这条原理重要：它把「我们需要几个架构视图」这个永远吵不清楚的问题，转化成了「我们的相关方是谁、他们关心什么」这个可以具体讨论的问题。前者是方法论之争，后者是项目分析。

2.5 架构是对不可逆决策的提前投资

原理陈述：架构的经济学本质是——识别哪些决策不可逆（早做，多投入），哪些决策可逆（晚做，保留选择权）。好的架构不是「决定一切」，而是「保留在信息充分时再做决定的自由」。这里的关键经济学概念是**期权的价值**：一个决策如果现在信息不够，推迟它本身就是一种价值——等到信息充分时再做，比现在瞎猜然后返工要便宜得多。

决策类型	特征	操作
高不可逆	影响半径大 + 信息成本高 + 修改风险高	早做，充分分析，多人审查
中不可逆	影响半径中等	做初步分析，预留调整空间
低不可逆	修改成本低，可随时改对	快速决定，错了再改

为什么这条原理重要：它提供了架构投资的决策准则——不是所有架构决策都值得深度分析。把分析资源集中在真正不可逆的决策上，而不是均匀撒在每一个选择上。提前决定服务边界（因为改这个很贵）；推迟具体缓存策略的选择（因为现在信息不够，改这个也不贵）；决定数据所有权归哪个域（因为一旦错了数据迁移成本极高）——三类决策，三种投入。

三、架构的目的与意义

前两章定义了架构是什么、架构决策是什么、以及几条基本原理。本章追问一个更深层的问题：**架构到底有什么用？为什么值得投入？**从认知、决策、变更、组织、经济五个层面展开。

3.1 认知层面——架构是系统的心智模型

一个人同时能有效处理的信息量是有限的（经典是 7 ± 2 个组块）。一个真实的企业系统可能有几百个服务、几千张表、上万个接口——远超单个人的认知能力。

架构做的事情，本质上就是**压缩信息**：

原始复杂度：500 个服务 × 200 个接口 × 50 个数据对象 = 不可管理
架构压缩后：12 个业务域 × 每域 5-8 个核心构件 × 域间 3-5 个接口 = 可管理

这不是丢失信息——丢失意味着你无法还原。架构压缩是**保真压缩**：

- 你不需要同时记住 500 个服务的调用拓扑，但你需要知道「订单域→库存域→物流域」这三个域之间的接口契约
- 当一个需求变更请求出现时，你不需要遍历所有 500 个服务，而是先定位域→定位构件→再深入细节

一个好的架构，让你在处理任何具体问题时只需要关心系统的一小部分，而不用把整个系统装进脑子。

从组织层面看，一个没有架构描述的系统，每个人脑子里对系统的理解都不一样。架构提供了**共享心智模型**（shared mental model）：新成员加入时，架构是最有效的入门材料；跨团队协作时，架构定义了各团队的责任边界；决策讨论时，架构提供了共同的语言。

在一条从需求到实现的设计链中，每一层都是对上一层信息的保真压缩——信息不丢失，但组织得更可管理：

层间过渡	压缩方式
OR → STR	相关方需求将零散的原始需求按角色归并，形成结构化的 What
STR → BA	BA 将需求抽象为输出文档（定义各文档的活动节点和正文条目），压缩掉实现细节
BA → SR	SR 将输出文档转化为系统功能点，从业务视角切换到系统职责视角

层间过渡	压缩方式
SR → PA	PA 将系统功能点归并到可实现的构件，从逻辑设计压缩为物理设计

3.2 决策层面——架构划定解空间

架构的作用不是给出所有答案，而是**划定解空间**——哪些方案是可以接受的，哪些不是。

无架构：解空间 = 无穷大 → 选择瘫痪
有架构：解空间 = 架构约束下的有限空间 → 可管理的设计决策

举例：如果在 PA 层定了「订单处理」是一个独立服务（架构决策），那么后续所有关于订单的详细设计都在这个框里做——你不需要在每次加订单功能时重新争论要不要拆服务。**架构削减的不是工作量，而是决策量。**

这直接关联到 §2.3（约束即解放）和 §2.5（不可逆决策提前投资）——架构约束消除了低价值的决策空间，让你把注意力集中到真正需要创造力的地方。

3.3 变更层面——架构是系统的免疫系统

系统不会静止。需求变、业务变、技术栈变、人员变。**架构不是为静态世界设计的，而是为持续变化设计的。**

架构在变更管理上的作用从三个时间维度来看：

a) 事前——预判变更的影响范围。 你在一条需求上改了两个字：「订单确认后**同步**通知仓库」→「订单确认后**异步**通知仓库」。没有架构：需要翻遍代码找到所有通知仓库的地方，漏一个就是一个线上事故。有架构：追溯矩阵告诉你 OR-042 → STR-F-012 → BA-节点 → SR-F-031 → PA-订单服务 + PA-消息中间件。立刻知道要改哪两个构件，以及变更影响波及到哪些下游节点。

b) 事中——吸收变更而不崩溃。 不是所有变更都应该触发级联重设计。架构应该定义变更的刚性边界 vs 柔性区域：

变更类型	影响范围	架构应该提供的支持
添加新的需求条目	新增整条映射链路	架构应支持插拔式新增，不破坏现有结构
修改现有需求指标值	仅影响层级链中承接该指标的节点	指标分解的可追踪链路
新增一个业务域	可能影响顶层分类和多个下游节点	域间松耦合，新域不破坏旧域
更换底层技术组件	PA 构件内部实现变更，不影响上层	PA 构件的封装性——内部可替换而不改变接口契约

c) 事后——从变更中学习。 每次架构层面的变更失误都应该沉淀为架构知识的更新——「为什么这个接口契约的设计没有预见到这种使用场景？」→ 下次设计类似接口时，这个教训成为架构原则

的一部分。这就是架构定义中「演化的原则」的真正含义——架构不是一张图，而是一个**学习系统**。

3.4 组织层面——架构是团队的组织原则

康威定律（Conway's Law）：

任何组织设计出来的系统，其结构都是该组织沟通结构的副本。

反过来也成立——**逆康威定律**：

如果你想让系统有某种结构，就先让团队有对应的组织结构。

架构在这里扮演的角色是**协调层**。具体来说，产品架构（PA）层的构件边界就是天然的团队边界：

- 如果 PA 定义「订单服务」和「库存服务」是两个独立的构件，它们可以由两个团队独立开发、独立测试、独立部署
- 如果订单和库存被定义为一个构件的两个子模块，那它们大概率在同一个团队里

这就是为什么 PA 构件应满足**三独立**——可独立分配、独立替换、独立验证。这三个「独立」不只是技术属性，也是**组织属性**：可独立分配 = 可以分配给不同团队；可独立替换 = A 团队重构订单服务不影响 B 团队。

3.5 经济层面——架构是一种投资

架构决策的修改成本不是线性的。这里有两个关键经济学概念：

概念 1：不可逆决策的提前锁定。 有些决策一旦做了，修改成本随时间指数增长——数据所有权归属、服务边界、对外 API 契约。这些需要在信息最充分的时候做，并投入足够的分析。

概念 2：期权的价值。 有些决策可以推迟——等到信息更充分时再做，比现在瞎猜然后返工要便宜得多。好的架构师知道哪些决策需要现在锁死、哪些可以留给未来。

提前决定：服务边界（因为改这个很贵）

推迟决定：具体缓存策略（因为现在信息不够，改这个也不贵）

必须早做：数据所有权归哪个域（因为一旦错了，数据迁移成本极高）

3.6 架构的本质——系统的基本概念

回到 ISO 42010 的定义中一个容易被忽略的关键词——「基本概念或属性」。它不是在说架构的**内容**（元素+关系+原则），而是在说架构的**本体**：

架构不是你画的图、写的文档。架构是系统的「基本概念」——那些决定系统是什么而非其他什么东西的本质属性。

图可以过时，文档可以过时，但如果架构决策本身是正确的（数据所有权按业务域划分、构件间通过明确接口通信、变更通过追溯链路评估影响），那么这些「基本概念」会持续发挥作用，不管当初画的那张图还有没有人维护。

软件思想家 Ralph Johnson（《设计模式》四位作者之一）有一个精辟的表述：

Architecture is about the important stuff. Whatever that is.

这有一个深刻的隐含前提：**「什么是重要的」本身是需要判断的，而判断依据就是你对这个系统的理论。**

所谓「系统的理论」，是你对这个系统的核心运作逻辑的基本判断——它回答三个问题：这个系统靠什么成功？（定位核心价值驱动力）它会面临的¹最大压力在哪里？（识别关键约束点）什么东西做错了最致命？（区分轻重缓急）

举一个具体例子。同样是做一个电商平台：

	架构师 A 的理论	架构师 B 的理论
核心判断	这个平台靠订单吞吐量取胜——双十一峰值是日常的 50 倍，撑不住就没生意	这个平台靠商品发现取胜——用户不是因为没地方下单而流失，而是因为找不到想要的
"什么东西重要"完全不同	订单管道的水平扩展、数据库分片策略、消息队列的削峰能力	搜索索引的实时性、推荐管线的延迟、A/B 实验框架的灵活性
"什么东西可以推迟"完全不同	复杂的推荐算法可以先做一个简单的热榜凑合用	极致的吞吐量可以先靠加机器撑过去，优化的事以后再说

两个架构师面对的是同一个电商系统，但因为对"这个系统靠什么成功"的基本判断不同，他们画的架构图、做的架构决策、优先投入的资源方向完全不同。**这就是"理论"在起作用。**

一个没有理论支撑的架构就像没有物理学原理的工程——你可能碰巧做对，但不知道为什么对，下次换一个上下文就错。而一个好的架构师，不是那个能画出最漂亮架构图的人，而是那个**对自己系统的理论最清醒的人**——他知道自己的判断依据是什么，因此也知道在什么条件下这个判断会失效。

3.7 目的与意义总结

维度	架构的作用
认知	压缩信息，提供共享心智模型——好的架构让你每次只需关心系统的一小部分
决策	约束解空间，消除低价值选择——架构削减的不是工作量，而是决策量
变更	预判影响、吸收变更、从变更中学习——架构是系统在时间维度上的免疫系统
组织	定义团队边界和协作接口——架构是团队的组织原则（康威定律）

维度	架构的作用
经济	用前期分析成本换取后期返工减少，保留可推迟决策的期权价值

一句话收束：

架构是对不可逆决策的提前投资，目的是在系统生命周期中最大化每个决策的认知质量，同时最小化每个决策的修改成本。

四、架构决策

第一章定义了架构决策是什么，第二章和第三章阐述了它为什么重要。本章展开讨论两个操作性问题：怎样判断一个具体决策的不可逆性？以及 怎样记录架构决策？

4.1 不可逆性的三层判断框架

「不可逆」不等于「技术上无法修改」。几乎任何软件决策理论上都可以修改。「不可逆」的真正含义是：修改成本已经高到一个临界点，以至于理性的决策者会选择「忍受现有架构的不完美」而不是「掏出修改成本」。

判断一个架构决策的不可逆性，从三个维度分别评估：

第一层：修改的影响半径

改这个决策，需要联动修改多少东西？影响半径决定了修改成本的下限。

维度	低耦合（易修改）	高耦合（难修改）
结构耦合度	数据归属明确，改一个域不影响其他域；内部接口，单一消费者	数据跨多个服务共享，改所有权涉及数据迁移；公开 API，几十个外部消费者
时间耦合度	可渐进式迁移——v1 接口保留 3 个月，下游逐步迁移到 v2	必须一次性同步上线——如数据库 schema 变更，所有读写方必须同步升级
组织耦合度	修改只需一个团队内部协调	修改需要跨团队对齐、同步排期。技术上的耦合可以通过工具解决，组织上的耦合需要开会

第二层：修改的信息成本

有些决策修改的成本不在「动手改」，而在「重新搞清楚应该改成什么样」。

决策的知识深度：当初这个决策是靠一条长推演链做出来的——跨多层、多域分析权衡后才得出的结论。修改它意味着需要**重走一遍推演链**，重新确认所有下游依赖仍然成立。追溯链路越长，修改的信息成本越高。

决策的隐含上下文：当初做决策的背景信息没有被记录下来，只在当事人的脑子里。当这些人离开、忘记或转岗后：

修改一个决策的真正成本 = 重新发现当初为什么做这个决策的成本 + 实际修改的成本

决策记录（\$4.2）的价值正在于此——降低未来的信息恢复成本。

第三层：修改的风险

有些决策不是不能改，而是一旦改错了后果严重到无法接受。

爆炸半径：被修改的东西是否处于业务关键路径、数据关键路径、安全关键路径的交汇点？

修改内部管理后台的 UI 颜色 → 错了 → 难看，不急
修改核心支付流程的状态机 → 错了 → 资金损失，紧急
修改用户认证的鉴权模型 → 错了 → 安全漏洞，灾难

数据完整性风险：涉及历史数据迁移的修改自动升级一个风险等级：

修改新写入数据的 schema → 低风险（新的按新结构写）
修改已有历史数据的 schema → 中风险（迁移脚本可能出错、可能丢数据）
修改多服务共享的核心数据所有权 → 极高风险（数据迁移 + 多服务同步上线 + 回滚策略复杂）

这就是为什么**数据所有权的划分**几乎是架构中最不可逆的决策——一旦数据被多个服务依赖，修改所有权就像在飞行中给飞机换引擎。

综合判断矩阵

决策示例	影响半径	信息成本	修改风险	综合判定
服务边界划分	大（影响团队组织+接口契约）	高（涉及业务域理解）	高（数据迁移）	高不可逆
对外 API 接口契约	大（外部消费者不可控）	中	高（破坏外部集成）	高不可逆
数据所有权归属	大	高	极高（数据迁移）	极高不可逆
架构模式（微服务 vs 单体）	极大	高	极大	几乎不可逆
数据库选型	大	中	中（有迁移工具但成本高）	中-高不可逆
编程语言选择	大（全代码库）	低（社区标准明确）	中（重写成本高但可渐进）	中不可逆
内部工具库选型	中	低	低（可逐步替换）	低不可逆

决策示例	影响半径	信息成本	修改风险	综合判定
日志格式	小	低	低	高度可逆

4.2 架构决策的记录格式

一个架构决策的完整记录，回答六个问题。下面用一个真实例子来演示——某个 EOS 流水线项目中关于系统设计子线按四种场景模式组织的决策：

决策名称：系统设计子线按四种场景模式（瀑布/敏捷/逆向/DevOps）组织，而非统一流程

背景：EOS 流水线的系统设计子线需要覆盖不同的设计场景——有的是全新系统（需求明确），有的是迭代开发（需求渐进），有的是补文档（已有系统），有的是小变更（快速交付）。一种流程无法同时满足四种场景的效率要求。

选项：

- A. 单一流程，通过参数/配置来适配不同场景
好处：简单，一套文档规范
代价：对 DevOps 场景太重（几小时的活走 30 天流程不合理），对瀑布场景又太弱（缺少足够的分析深度）
- B. 四种独立模式，各自有独立的任务定义和操作流程 ← 选了它
好处：每种场景匹配最合适的分析深度和周期
代价：需要维护四套任务定义，四种模式之间的切换需要显式决策

选择：B。因为场景之间的效率差异太大（几小时 vs 30天），一套流程无法兼顾——用错了流程的代价 > 维护四套的代价。

影响：系统设计子线下有了四个独立子目录（瀑布/逆向/敏捷/DevOps），每个 Skill 需要定义四种模式的变体或注明适用模式。后续新增设计场景时，需要判断是归入现有模式还是新增模式。

不可逆性：高。因为所有下游 Skill 的任务定义都建立在四模式分类之上，如果要改成三模式或五模式，需要重写所有 Skill 的操作流程。

这个六问格式——决策名称、背景、选项、选择、影响、不可逆性——适用于任何架构决策，无论它发生在需求组织层（STR）、业务行为层（BA）、系统功能层（SR）还是产品构件层（PA）。

4.3 不可逆性判断的操作化落地

不可逆性判断不只是理论——它可以嵌入到设计过程中，转化为具体的操作机制。

4.3.1 不可逆性标注

在架构末级节点上标注其不可逆性，使后续维护者能一眼识别修改风险：

PA-订单服务：

不可逆性：高

理由：数据所有权归订单域，涉及订单→库存、订单→支付的接口契约

修改代价：影响 3 个域 × 5 个服务 × 2 个外部系统集成

标注的价值不在当前设计者——当前设计者知道为什么这么选。价值在**一年后换了一个人接手**：他看到「不可逆性：高」就知道不要轻易动这个决策，看到「理由」就知道动了会波及什么。

4.3.2 按不可逆性分级的审查深度

不是所有架构决策都值得同样深度的审查。按不可逆性调整审查投入：

不可逆性	最低审查深度	审查参与者
极高	语义+指标完整审查	多人交叉审查
高	语义+指标完整审查	至少一人独立审查
中	语义符合性	自查 + 同行确认
低	快速自检	自查即可

这个分级审查机制确保**把有限的审查资源集中在真正不可逆的决策上**——而不是均匀撒在所有决策上。

4.3.3 「推迟决策」作为一种显式架构操作

不是所有决策都需要立即做出。当一个决策的不可逆性高但当前信息不充分时，**「明确决定推迟」本身就是一个有效的架构操作**。

这在快速迭代场景中尤其重要。例如 DevOps 模式通常倾向于快速决策，但并非所有变更都是低不可逆的：

DevOps 模式标准流：

收到小变更 → 快速分析 → 实施

嵌入不可逆性预判后：

收到小变更 → 不可逆性预判（30 秒）→

低不可逆 → 快速实施

高不可逆 → 标记为「需完整设计推演」→ 升级到更重的分析路径

架构师的核心能力不是「做出正确的决策」，而是「识别哪些决策如果不正确后果很严重」——对不重要的事情不决策，对信息不足的高风险决策主动推迟，本身就是好的架构决策。

五、架构视图

本章从原理 2.4（视图完备性由相关方决定）出发，展开讨论如何系统性地推导一个系统需要哪些架构视图。

5.1 主流框架对比——为什么没有标准答案

历史上几乎所有架构框架都试图回答「需要几个视图」，它们的答案各不相同：

框架	几个视图	是哪几个	面向什么场景
Kruchten 4+1 (1995)	5	逻辑 + 进程 + 开发 + 物理 + 场景 (+1)	单个软件系统
Rozanski & Woods (2005)	7	上下文 + 功能 + 信息 + 并发 + 开发 + 部署 + 运维	单个软件系统 (比 4+1 更 细致)
TOGAF (最新版)	4A	业务 + 应用 + 数据 + 技术	企业级 IT 资产管理
DoDAF (美国国防 部)	8 类 52 个	全视点 / 能力 / 数据与信息 / 作战 / 项目 / 服务 / 标准 / 系统	巨型系统体系 (武器+指挥 +后勤.....)
ISO 42010 (2011)	不定	不规定——只规定「怎么从相关方推 导视图」	通用

每个框架的数字不一样，而且跨度极大（4 到 52），但没有一个框架说「6 个」或者「8 个」是标准答案。这不是因为某个框架对、某个框架错，而是因为每个框架面向不同类型的系统、不同类型的企业、不同类型的相关方。DoDAF 有 52 个视图是因为它面对的是美国国防部的相关方离散度——国会（出资方）、各军种（使用者）、承包商（构建者）、测试评估机构（鉴证者）、盟军（集成方）——每一方都需要用自己能理解的语言看到关心的内容。

正确的思维方式不是「挑一个框架的视图清单来用」，而是**从自己的相关方出发重新推导**。

5.2 推导方法——从相关方到视图的三步法

第一步：识别相关方及其关注点

这是绝对不能跳过的一步。相关方是谁，不是方法论告诉你的——是你通过分析具体项目得出来的。

但有五类相关方由「系统从概念到运行到退役的完整生命周期」决定，**几乎必然出现**。不是因为方法论规定了它们，而是因为系统存在的逻辑本身就隐含了这些相关方：

相关方类别	为什么必然存在	核心关注点
采办者 (Acquirer)	有人出钱/出资源让系统存在	系统能解决我的问题吗？花多少钱、多少时间？
使用者 (User)	系统存在的唯一目的是被使用	我怎么用？能帮我做什么？好用吗？
构建者 (Builder)	抽象设计不会自己变成运行的系统	我要做什么？边界在哪？依赖什么？
运维者 (Operator)	任何系统运行就需要被运维	怎么部署？怎么监控？故障如何恢复？容量够不够？

相关方类别	为什么必然存在	核心关注点
鉴证者 (Assurer)	系统存在于一个有约束的环境中（法规/标准/合规/安全）	安全吗？合规吗？审计能过吗？违反法规谁担责？

关于“鉴证者”——“鉴证”= 先**鉴**（独立审视系统是否满足外部标准），后**证**（给出认定结论）。这个角色不建造、不运维、不使用——只做检查和认定。消防检查官就是典型的鉴证者。

这五类相关方不是「选择的」，是「识别出来的」。你可能在某些项目里两类合并成一个人（比如构建者也运维），但关注点本身不会消失。

为什么没有“物料供应方”？

五类的划分逻辑是**生命周期角色**——每个角色对应系统从概念到退役的一个必然阶段：被采办（采办者）→ 被设计/建造（构建者）→ 被使用（使用者）→ 被运维（运维者）→ 被鉴证（鉴证者）。物料供应方不在这条生命周期链上——它在**供应链**上，是构建者的上游，对系统的关切是“我的产品被正确使用”而非“这个系统的架构是否合理”。

但物料供应方作为一个**条件性相关方**是成立的：当系统严重依赖某个特定外部组件——专有协议、独家硬件、特定云服务——且该组件的演进方向会制约架构选择时，供应方应纳入相关方分析。关键在于：不是“所有供应商都算”，而是“供应商的决策会影响你的架构决策时，才算”。

第二步：将关注点聚类为视图

每个相关方有一堆关注点。但**不是每个关注点都需要一个独立的视图**——那样视图数量会爆炸。需要按两个原则聚类：

原则 1：同一视图内的关注点，可以用同一套语言、同一种表达方式来回答。比如使用者的「我怎么做采购下单？」和采办者的「系统支持了哪些业务流程？」——都可以用业务场景叙事来回答，可以放在同一个视图里。但使用者的「我怎么做采购下单？」和运维者的「数据库怎么备份？」——需要完全不同的语言，必须放在不同视图里。

原则 2：视图的数量上限，由人类同时能有效处理的视角数量决定。如果一个架构描述有 15 个视图，没人能看完——它**实际上不会被人使用**，等于不存在。经验上：3-5 个视图绝大多数人能有效理解和维护；6-8 个需要架构团队专业维护，相关方按需查阅；9+ 个通常意味着聚类不够，存在冗余。

第三步：检查完备性

聚类之后，用两个问题自检：

1. 有没有重要相关方的核心关注点，在任何一个视图都找不到答案？
→ 如果有，缺视图，需要增加或扩展现有视图。

2. 有没有视图的内容，不被任何重要相关方需要？
→ 如果有，这个视图是多余的，删掉或合并。

5.3 七候选视图

用三步法完整走一遍，对于绝大多数业务系统/软件产品，答案会收敛到七个候选视图：

候选 1：系统上下文视图 (Context View)

维度	内容
回答什么	系统在哪？边界在哪？和外界是什么关系？
目标相关方	所有相关方（这是理解系统的起点）
核心表达	系统边界图 + 外部实体清单（用户、外部系统、环境）+ 交互关系
可省略性	几乎总是需要。没有边界就没有系统

候选 2：用户操作视图 (User Operational View)

维度	内容
回答什么	用户怎么使用系统来完成他们的工作？哪些角色、在什么场景、执行什么操作？
目标相关方	使用者、采办者
核心表达	用户角色与职责 + 操作场景（用户故事/用例）+ 用户端到端工作流 + 用户与系统的交互序列
可省略性	几乎总是需要。采办者通过它验证「系统能不能支撑业务」，使用者通过它确认「我的工作怎么做」

为什么需要一个独立视图？ 系统上下文视图说清了「谁在外面、怎么连着」，但不回答「外面的这些人具体怎么用系统做他们的事」；功能/行为视图的主语是系统（系统提供什么、系统内部怎么协作），不回答用户的操作流程。用户操作视图填补了这个空白——主语是**用户**，描述的是运行概念（Operational Concept）：用户按照什么流程、在什么场景下操作系统来达成自己的工作目标。

候选 3：功能/行为视图 (Functional/Behavioral View)

维度	内容
回答什么	系统提供哪些功能？这些功能之间如何动态协作？

维度	内容
目标相关方	构建者、使用者
核心表达	功能分解（功能树/功能分类）+ 功能间的动态交互（顺序、状态、活动）+ 功能与需求的映射
可省略性	几乎总是需要。系统对绝大多数相关方来说首先是「能做什么」

为什么不是“用户怎么使用它”？ “用户怎么使用”的主语是用户，描述的是用户的操作流程——那属于运行概念，应在用户操作视图中描述（见候选 2）。功能/行为的主语是**系统**——功能 = 系统的静态能力分解（系统有什么能力），行为 = 系统的动态协作模式（这些能力之间怎么一起工作）。

候选 4：信息/数据视图 (Information/Data View)

维度	内容
回答什么	系统管理什么数据？数据怎么组织、归谁管、怎么流动？
目标相关方	构建者、鉴证者（数据合规）
核心表达	领域数据模型 + 数据所有权 + 数据流
可省略性	系统的数据很简单、不共享、不需要建模时可合并到其他视图

候选 5：构建/开发视图 (Construction/Development View)

维度	内容
回答什么	系统由哪些构件组成？构件之间怎么交互？
目标相关方	构建者（开发者）
核心表达	构件清单 + 构件定义 + 构件接口契约 + 构件依赖拓扑
可省略性	几乎总是需要。这是开发者唯一能看到「自己该做什么」的视图

候选 6：部署/运行视图 (Deployment/Runtime View)

维度	内容
回答什么	系统跑在什么环境上？怎么部署、监控和维护？
目标相关方	运维者、构建者
核心表达	部署拓扑 + 基础设施需求 + 运行时依赖 + 监控/告警策略

维度	内容
可省略性	单机简单系统可合并到构建视图；分布式/云原生系统必须独立

候选 7：质量/约束视图 (Quality/Constraint View)

维度	内容
回答什么	系统要满足什么非功能约束？有哪些合规要求？
目标相关方	鉴证者、运维者、采办者
核心表达	NFR 分类清单 + 质量指标 + 合规要求 + 技术约束
可省略性	至少需要嵌入到其他视图中；质量要求复杂的系统（安全/性能/可用性/合规各自有独立相关方）必须独立

5.4 相关方 × 视图对应关系

一个相关方通常需要不止一个视图：

相关方	主要视图	辅助视图	备注
采办者	系统上下文 + 用户操作	功能/行为 + 质量/约束	采办者需要知道「系统能支撑哪些业务操作」和「系统边界在哪」
使用者	用户操作	系统上下文 + 功能/行为	使用者首要关心「我的工作怎么做」，其次才是系统能做哪些事
构建者	构建/开发 + 信息/数据	部署/运行 + 功能/行为	—
运维者	部署/运行 + 质量/约束	构建/开发	—
鉴证者	质量/约束	系统上下文 + 信息/数据	鉴证者需要一个地方一次性看到全部 NFR 承诺

5.5 决定视图数量的三个因素

因素	对视图数量的影响
系统复杂度	简单系统（单机工具、静态网站）3-4 个视图就够，部署和质量可合并到构建视图；中等系统（企业应用、SaaS）5-6 个视图；复杂系统（分布式平台）7 个视图且某些视图需细分——部署可能拆为运行时进程视图和物理部署视图，数据可能拆为概念数据模型和物理数据存储，质量必须独立

因素	对视图数量的影响
相关方离散度	所有相关方同属一个组织 → 视图可少，一门语言沟通；跨部门 → 每个部门至少需要一个看得懂的视图；跨公司（甲乙方、多方集成） → 每个外部相关方可能需要独立视图
生命周期阶段	概念阶段：上下文 + 功能（粗粒度）即可；设计阶段：需要全部但各视图详细程度不同；构建阶段：构建+部署最详细；运维阶段：部署+质量最活跃

5.6 完备性自检——不看数量看覆盖率

自检问题	对应视图
一个完全不懂技术的人，能不能看懂系统的边界和目的？	系统上下文视图
一个业务用户，能不能搞清楚自己在系统里的工作流程和操作场景？	用户操作视图
一个开发者，能不能搞清楚系统提供哪些功能、功能之间如何分类和协作？	功能/行为视图
一个新加入的开发者，能不能搞清楚系统有哪些构件、每个构件管什么、构件之间怎么交互？	构建/开发视图
系统管理和使用了哪些核心数据？数据归谁管、谁在用？	信息/数据视图
一个运维工程师，能不能知道系统跑在哪里、怎么部署、怎么监控？	部署/运行视图
一个安全审计者，能不能一次性看到系统的全部 NFR 承诺和合规要求？	质量/约束视图

如果上面某个问题的答案是「不能」——你的视图不够，不管你现在有几个。

5.7 追溯——贯穿全部视图的横切面

严格来说它不是第八个视图，而是贯穿全部七个视图的一条线——可追溯性。

任何一个相关方在任何一个视图中看到任何一个元素，都应该能回答：「这个东西是从哪里来的？它要满足谁的需求？」

追溯的目标相关方是**所有相关方**：业务相关方看到业务操作上的性能指标 → 追溯到需求来源 → 确认「这确实是我提的」；开发实施者看到构件上的接口定义 → 追溯到功能需求 → 确认「这个接口是为了满足那个功能」；系统设计者收到需求变更 → 追溯变更影响的全部构件 → 评估变更规模。

追溯不独立成为一个视图，但它是**所有视图之间的连接组织**——没有它，七个视图就是七张互不相关的图。

5.8 视图 vs 文档——一个关键区分

视图 ≠ 文档。一个视图可能由多个文档承载，一个文档也可能服务于多个视图。

视图是**逻辑概念**——它定义了「谁看什么」。文档是**物理概念**——它是承载内容的文件。二者的关系是 N:M：

- 系统上下文视图可能由「系统概述」一个文档承载，也可能由「相关方需求」「系统边界定义」「外部接口规范」三个文档共同承载
- 一个「产品架构文档」可能同时服务于构建视图（构件清单部分）和部署视图（部署拓扑部分）

混乱的根源往往是：**把物理文档的数量当成了逻辑视图的数量，或者反过来，要求每个逻辑视图都对应一个独立文档。**

5.9 视图与设计链的映射——以一个五层链为例

把七候选视图映射到一个具体的五层设计链（OR → STR → BA → SR → PA），可以看到每层产出对应了哪些视图：

七视图	对应的设计链产出	覆盖状态
系统上下文视图	OR + STR（系统边界、角色集、外部交互）	✅ 覆盖
用户操作视图	STR（用户角色）+ BA（业务场景、端到端工作流）	✅ 覆盖——用户视角的操作流程贯穿 STR 角色定义和 BA 业务场景
功能/行为视图	STR-F + BA + SR-F（需求→输出文档→系统功能）	✅ 覆盖，但分散在三层
信息/数据视图	BA（业务数据流）+ SR/PA（逻辑/物理数据模型）	⚠️ 部分覆盖——数据架构的视角分散在多层，无独立聚合点
构建/开发视图	PA（构件清单 + 构件定义 + 接口契约）	⚠️ 大部分覆盖——构件关系（接口契约、依赖拓扑）的显性化程度不统一
部署/运行视图	—	❌ 未显式覆盖——部署拓扑、基础设施需求、运行时约束在设计链中不在设计范围内
质量/约束视图	STR-NFR + SR-NFR（NFR 分类 + 指标）	✅ 覆盖——NFR 有独立平行路径，不与功能混淆
（追溯）	追溯矩阵	✅ 覆盖——贯穿全部视图的连接组织

两个值得关注的缺口：

缺口 1：部署运维视图的缺失。 设计链止于 PA 层，部署属于集成阶段，运维属于运维阶段。这本身没有问题——部署和运维的详细内容确实不属于「系统设计」。但系统设计阶段需要知道**部署运维对架构的约束**——比如「这个构件必须能独立部署」「这个数据必须支持跨数据中心同步」——这些约束如果不在系统设计阶段提出来，到了集成/运维阶段再发现就会变成架构返工。建议在设计

终点（PA 层）增加一个轻量的「部署运维架构约束」章节——不是完整部署手册，只需要回答：系统设计中有哪些决策是因为部署/运维约束而做出的。

缺口 2：构件关系的显性化程度。 构件之间的关系（接口契约、调用拓扑、数据流）往往比单个构件的定义更重要——因为系统的复杂性不在构件内部，而在构件之间。当构件间关系足够复杂时，可以将其从附属说明提升为架构定义的一个独立部分，如「构件关系视图」（包含接口契约矩阵 + 构件依赖拓扑）。

六、企业架构4A模型——为什么是这四个？

有了前五章的基础，本章专门回答一个困扰很多人的问题：**为什么企业架构（EA）翻来覆去就是「业务架构、应用架构、数据架构、技术架构」这四个？它们和架构视图是什么关系？**

6.1 4A的起源——不是设计出来的，是长出来的

TOGAF 的 4A (BDAT) 可以追溯到 1987 年 John Zachman 的框架，后来被 TOGAF 吸收并固化。它的形成逻辑不是「相关方需要什么视图」，而是「企业里有什么东西需要被管理」：

企业有什么？ → 有业务流程和业务能力 → 叫「业务架构」
→ 有应用系统 → 叫「应用架构」
→ 有数据 → 叫「数据架构」
→ 有技术基础设施 → 叫「技术架构」

这是按管理对象分类，不是按相关方关注点分类。

6.2 4A就是四个管理地盘

从 1987 年到现在，这四分类几乎没变过——因为它刚好映射到大型企业的组织架构：

EA 域	对应组织中哪个部门	谁在管
业务架构	业务部门、流程管理部	业务负责人
应用架构	IT 应用管理部	应用架构师
数据架构	数据管理部	数据架构师
技术架构	IT 基础设施部	基础设施架构师

4A 就是四个管理地盘。 4A模型本质上是一个**资产管理框架**——它回答「企业有哪些 IT 资产、每个资产归谁管」。而架构视图回答的是「这个系统的架构怎么让所有重要相关方理解」。两者服务于不同的目的。

6.3 4A × 七视图——两个分类维度的交叉

4A (按管什么切) 和七视图 (按给谁看切) 是两个独立的分类维度, 它们可以交叉:

	业务域	应用域	数据域	技术域
系统上下文视图	✓ 边界定义	✓ 系统交互	-	-
用户操作视图	✓ 核心覆盖	✓ 用户交互	-	-
功能/行为视图	✓ 核心覆盖	✓ 应用功能	✓ 业务数据流	-
信息/数据视图	-	✓ 数据模型	✓ 核心覆盖	-
构建/开发视图	-	✓ 核心覆盖	✓ 物理存储	✓ 技术选型
部署/运行视图	-	✓ 应用部署	✓ 数据部署	✓ 核心覆盖
质量/约束视图	✓ 业务 SLA	✓ 应用性能	✓ 数据质量	✓ 技术 SLA

4A是纵向切——把架构描述按管理归属分类。七视图是横向切——每个视图可能横跨 4A 的多个分类。

6.4 4A模型的三个问题

单独使用4A作为架构描述框架，存在三个结构性缺陷：

问题 1：需求视图被「业务架构」吞掉了。 在 TOGAF 里，需求管理是一个中心圆环放在 4A 的中间，但需求本身不被当成一等视图来对待。这导致需求只在项目初期被认真对待，然后就变成了一堆散落的文档，没有人持续维护它们和下游设计的关联。

问题 2：质量/NFR 散落在 4A 里，没人对它负责。 安全性在哪？在技术架构里（安全基础设施），也在应用架构里（应用安全），也在数据架构里（数据脱敏、权限），也在业务架构里（合规流程）。结果：安全性被 4A 各分类瓜分，没有一个视图能让相关方一次性看到「这个系统在安全上到底做了什么承诺」。质量要求需要一个独立的一等视图——不是 4A 各分类各自带一点。

问题 3：4A 各分类之间的动态关系丢失了。 4A 各分类各自是静态的快照。但架构中真正重要的东西——需求的变迁如何影响设计、业务场景如何转化为系统功能、一个构件的修改如何波及上下游——这些跨分类的动态关系在 4A 模型中没有被显式表达。

6.5 4A为什么这么流行

尽管有上述缺陷，4A模型依然占据统治地位。三个原因：

原因 1：TOGAF 认证体系锁定了话语权。 全球几十万 TOGAF 认证者，教材怎么写他们就怎么讲，考试怎么考他们就怎么信。

原因 2：它确实够用——在特定场景下。 当你问「公司有多少个应用系统？它们用哪些数据库？跑在什么服务器上？」——4A模型完胜。它是一个优秀的 IT 资产管理工具。但当你问「这个系统的架构是什么」——需要让业务方、需求方、设计者、开发者、运维者都理解系统——4A模型捉襟见肘。

原因 3：简化的力量。 四个概念，一张 PPT 放得下。业务/应用/数据/技术——谁都觉得自己能理解，不需要额外学习。你讲「七视图」需要解释每个是什么，你讲「4A」对方点头说懂了（虽然大概率理解不一致）。

6.6 4A与视图的共存——各司其职

	4A (BDAT)	七候选视图
目的	企业级 IT 资产的 管理和治理	一个系统架构的 理解和沟通
组织逻辑	按管理对象分类（谁管这个）	按相关方关注点分类（谁关心这个）
粒度	企业级（覆盖所有系统）	系统级（聚焦一个系统）
适用场景	IT 资产管理、技术路线规划、跨系统整合	系统设计、实现、部署、运维
谁在用	EA 团队、CIO、IT 治理委员会	系统设计者、开发团队、业务相关方

4A是「管资产」的框架。七视图是「描述系统」的框架。一个企业可能有 200 个系统——4A 帮你说清楚这 200 个系统的资产分布。但当你聚焦其中 1 个系统做设计时，七视图帮你说清楚这个系统的架构让所有人理解。

七、设计链中各层的架构决策

理解了架构决策（第四章）和架构视图（第五章）之后，本章聚焦一个具体问题：在一条从需求到实现的设计链中，每一层做的架构决策有什么不同？为什么不同？

以下以一条从原始需求（OR）→ 相关方需求（STR）→ 业务架构（BA）→ 系统需求（SR）→ 产品架构（PA）的五层链为例，逐层分析。注意：OR 是纯需求，无架构决策。架构决策从 STR 开始——因为 STR 是第一个「方案」，而方案才有架构。

7.1 为什么每层的架构决策不同

设计的本质是对需求的持续分解、分配和符合性分析，直至产出可实现的方案。在这个过程中，每一层面对的需求抽象层次不同、向下传递的目标对象不同，因此每层做的架构决策天然不同。

可以用一个医学类比来理解：

层	类比	在做什么决策
OR	患者说「我头疼、吃不下饭、睡不好」	不决策，只是表述
STR	医生分诊：头痛→神经内科，吃不下饭→消化内科，睡不好→两个科室都会诊	把散乱的自述归类到科室
BA	神经内科医生：偏头痛还是紧张性头痛？什么时候发作？持续多久？	把笼统主诉拆成可诊断的症状单元
SR	确诊并开检查单：偏头痛，做脑电图+MRI	把症状转化为可检验的医学判断
PA	开药方：普萘洛尔 40mg bid + 布洛芬按需	把判断转化为可执行的处方

每一层都在**承上启下**——把上一层的输出转译为下一层能理解和操作的语言。转译过程中必然要做决策。

但有一个贯穿各层的共性：**每一层的架构决策，本质上都在回答「这部分需求，由谁来承接、以什么形式承接、和其他承接者是什么关系？」**只不过每一层的「承接者」不同——STR 层的承接者是需求分类节点，BA 层的承接者是输出文档，SR 层的承接者是系统功能点，PA 层的承接者是产品构件。

7.2 需求有架构吗？——「需求方案」的架构

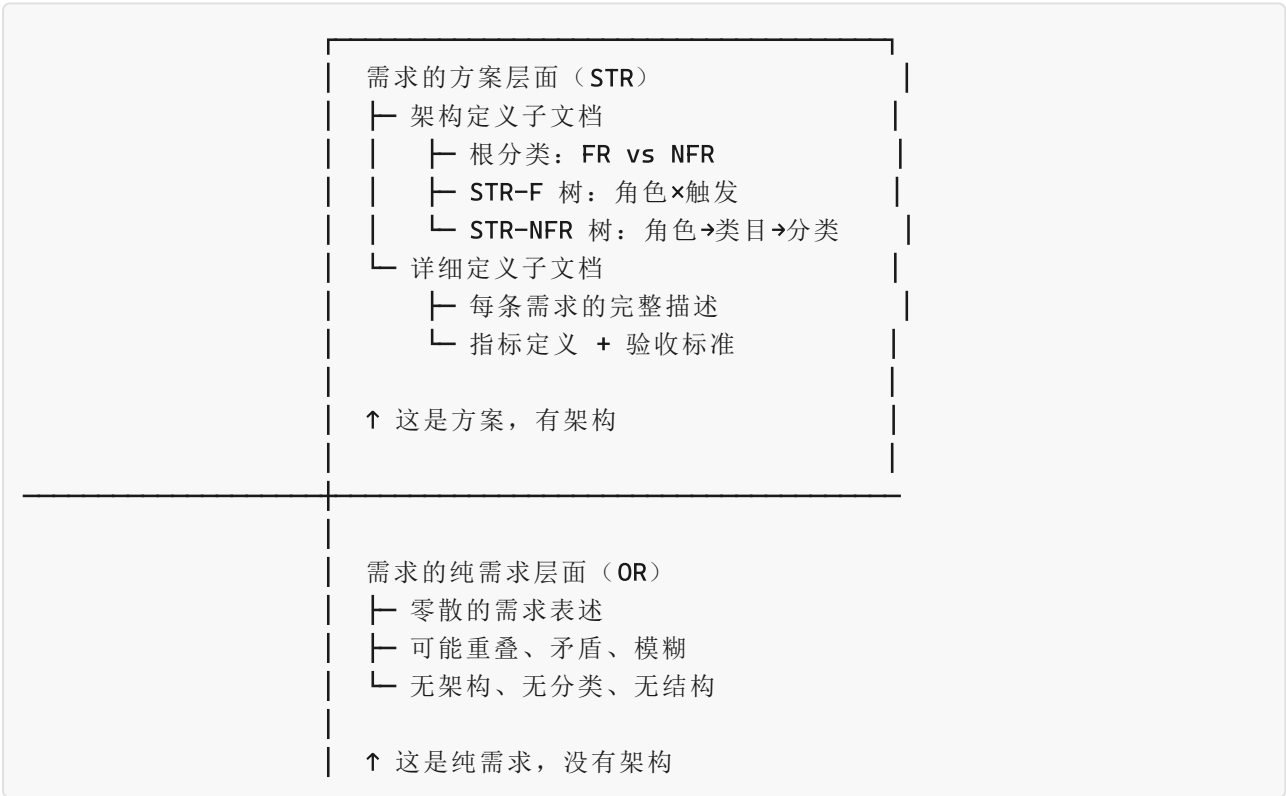
这是一个经常引起困惑的问题：「需求架构」这个词听起来像个矛盾——需求不是回答 What 吗？架构不是回答 How 吗？那需求的架构是什么东西？

7.2.1 答案：需求本身没有架构，但需求的方案有架构

OR（原始需求）是设计链的起点——它是一堆零散的、可能互相重叠甚至矛盾的需求表述。OR 没有架构定义子文档，没有详细定义子文档，只有条目清单。**纯需求没有架构，也不需要架构。**

但当你面对这堆零散需求，要把它们组织成一套结构化的、可验证的、可向下传递的需求体系时（STR），这个组织工作本身就是一次设计——它的输出是**需求的方案**，而方案就需要架构。

不存在「需求的架构」。存在的是「需求方案的架构」——即如何将零散的原始需求组织成结构化需求体系的架构。



OR 没有架构定义子文档，STR 有——这不是偶然的。**OR 是设计链的起点，它必须先存在，才有后面的事；STR 是第一个「方案」——它是「如何组织需求」这个设计问题的答案，而答案需要架**

构。

7.2.2 为什么这个区分重要

这澄清了一个深层的概念混淆。日常说「需求架构」时，实际指的是 STR 层的架构——即需求方案（STR）的架构，而非需求本身的架构。STR 层的架构决策（按什么维度分类、按什么颗粒度组织、冲突怎么处理）**不决定「系统怎么做」，但决定了「系统设计从哪里出发、按什么逻辑组织、怎么验证完备性」**。这些决策的影响贯穿整个设计链，不可逆性极高。

这一点和 §2.1（需求/方案相对）形成呼应：同一份文档在不同层位上兼具不同身份——STR 向上是 OR 的方案，向下是 BA/SR 的需求。这种双重身份正是「需求方案也需要架构」的机制基础。**STR 之所以既有架构定义子文档又有详细定义子文档，正是因为它在设计链中扮演着双重角色——作为方案需要架构来组织，作为需求需要详细定义来传递。**

7.2.3 STR 层的架构决策——一个更深的认识

STR 层的架构决策虽然不涉及任何系统实现细节，但它定义了**系统的概念边界**——系统要解决谁的问题（用户角色）、解决什么问题（需求内容）、做到什么程度算成功（指标）。它不是系统内部的架构，但它是系统的「外部架构」——系统与它的环境（用户、相关方、约束条件）之间的关系结构。

这回到了 §1.1 中「系统」定义四个要素之一——**边界**。STR 层做的架构决策，本质上就是在定义系统的边界：什么东西在系统里（需要被后续层设计）、什么东西在系统外（是需求来源、是约束条件、是不需要系统管的）、系统和外界的交互点在哪。从这个意义上说，**STR 层是整个五层链中唯一同时面对「系统内」和「系统外」的层——它既承接了外部相关方的需求输入，又为后续层的内部设计划定了起跑线。**

7.3 STR 层——定义「需求怎么组织」

这一层在解决什么设计问题：一堆零散的、可能重叠或矛盾的原始需求表述（OR），需要被组织成一套结构化的、可验证的、可向下传递的需求体系。

用一个具体例子来理解——假设你做了需求调研，带回来 100 张便签纸：

"订单提交后要能实时看到状态"、"系统不能太慢"、"财务那边要能导出报表"、"审批流程要支持多级"、"数据要安全"、"操作界面要简洁"、"要支持手机端操作"、"库存不足时要能自动提醒"……（还有 92 条）

你现在要做的就是把它们变成一套可以往下传递的需求文档。你立刻面临一系列决策：

决策 A——按什么维度分类？按「谁提的」（采购部/销售部/财务部）？还是按「功能 vs 非功能」？还是按「业务流程阶段」？如果你选了按提需求的人分类，同一条端到端流程会被拆到三个部门的需求堆里——采购员提了「下单要快」、审批者提了「审批要有超时提醒」、仓库提了「发货要能扫码」——这三条其实串在同一条流程上，但按部门分类让你看不出来。

决策 B——每条需求写到什么粒度？「系统不能太慢」是一条还是多条？拆成「订单列表页加载 < 2s」「订单提交接口 < 500ms」「审批流转 < 1s」——太粗了下游不知道要优化哪里，太细了需求决策者验证不了（「我怎么知道审批流转是不是 1 秒？我自己用起来不慢就行了」）。

决策 C——冲突的需求怎么组织呈现？ 采购部说「审批要严格，至少三级」，销售部说「审批要快，最好一级过」。两条都保留标注冲突等待裁决？还是预先合并为「审批支持灵活配置 1-3 级」？前者保持了需求原样但给下游增加了处理负担，后者简化了下游但提前做了设计决策（这可能不在需求阶段的权限范围内）。

这三个决策——分类维度、粒度约定、冲突处理规则——**它们都是架构决策，虽然不决定系统怎么做，但决定了整条设计链从哪里出发。**

STR 层的形式化架构决策：

决策	内容	备选方案	为什么架构决策
顶层按什么切分	FR/NFR 分轨	FR 和 NFR 混在一棵树上	NFR 独立分轨可以独立验证完备性，混在一起容易在功能需求中淹没非功能关注点
STR-F 按什么组织	用户角色 × 触发	按业务流程阶段 / 按业务模块 / 按需求来源部门	角色穷举 = 完备性可验证；其他方式做不到
STR-NFR 按什么组织	角色 → 一级类目 → 二级分类	只按分类不按角色 / 只按角色不按分类	角色保证相关方可验证，分类保证技术完备性——两面缺一不可
末级节点指代什么	STR-F：一个场景业务（端到端业务闭环），即一个完整的「某角色在某触发下做某事的端到端旅程」；STR-NFR：一个非功能特性维度	功能操作 / 业务模块	太细了下游 BA 的 N:1 合并爆炸，太粗了撑不起需求分配。判定标准：业务人员能否描述出一个完整的工作闭环？
冲突需求怎么处理	保留原样，标注冲突，等待裁决	预先合并 / 直接丢弃	STR 层不越界做方案设计决策；冲突裁决是相关方的权利

这一层的独特特征：STR 层是设计链的起点——它的上方没有可追溯的需求文档。完备性只能靠方法论保证（角色集穷举），不能靠追溯验证。**STR 的每一个架构决策，本质上都在回答：「我们怎么知道需求说全了？」**

不可逆性：极高。不是因为改文档的成本高，而是因为它是整条链的根——根歪了，整棵树歪。

7.4 BA 层——定义「输出文档的颗粒度」

这一层在解决什么设计问题：将功能需求从「用户要做什么」转译为「业务上需要产出哪些输出文档」。BA 不创造新的需求内容——全部输入来自 STR-F 的详细定义。它做的是转译，转译的质量取决于输出文档颗粒度的判断。

BA 层最关键的一个决策——输出文档的颗粒度：

- 太粗了：
输出文档：「采购管理」——一个笼统的业务领域
→ 这等于没分解。下游 SR 无法承接。
- 太细了：
输出文档：「供应商名称字段」
→ 这只是一份输出文档中的某个正文条目（PL6-B 级别），不是一份完整的业务输出。
→ 业务上没有人会说「我今天工作的产出是供应商名称字段」——它只是产出物的一个组成部分。
→ STR 的端到端业务有 30 个，拆成 300 个字段级别节点会让 SR 层的 N:1 合并量爆炸。
- 刚好：
输出文档：「采购订单」——一份完整的业务输出，业务人员可以指着它说「这就是我这一步要产出的其内部结构（活动节点、正文条目）属于 PL6 级别的详细设计，不在此层的架构决策范围内。

输出文档颗粒度的判定标准：一个业务人员能否指着这个节点说「对，这就是我工作中的一步，它产出的是一份完整的业务文档」？能 → 刚好。不能（「这太细了，只是一个文档里的字段」）→ 太细。能但一个节点对应了多个不同的文档（「你说的是一整套流程里的所有文档吧」）→ 太粗。

BA 层的形式化架构决策：

决策	内容	为什么架构决策
顶层按什么组织	业务域（按业务职能领域划分——采购、生产、销售、售后……）	业务域是企业组织结构的自然映射，域内高内聚、域间低耦合
末级节点指代什么	输出文档——一份完整的、可独立被业务人员讨论和验证的业务输出（如一张采购订单、一份审批单）。其下可按需拆分为活动节点（处理流程中的步骤序列）和正文条目（正文中满足独立定义条件的条目类型）	颗粒度决定 SR 层承接效率。判定标准：业务人员能否指着这个节点说「对，这就是我工作中要产出的文档」？
BA 节点怎么推导	用户角色架构锚定法（锚点在角色集）/ 输出产品架构锚定法（锚点在输出产品 PA 节点类型）	锚点决定完备性保证方式
要不要详细定义	不要。BA 只有架构定义，无详细定义	BA 是桥——把功能需求转化为输出文档后直接交给 SR 做视角转换，不需要再展开一层

这一层的独特特征：BA 不创造新内容，它做的是转译。转译的质量取决于输出文档颗粒度的判断，而颗粒度判断严重依赖领域知识。

不可逆性：中-高。修改成本在文档层面，但输出文档颗粒度决定了 SR 层桥接重组的起点。

7.5 SR 层——定义「系统功能的组织方式」

这一层在解决什么设计问题：将输出文档（BA 层的产出）转化为系统功能需求（SR-F），将分散的非功能关注点归并为统一的技术约束（SR-NFR）。这是设计链中**第一次出现「系统内部」概念**的层——在此之前，STR 和 BA 都在描述「系统外面有什么、业务上要产出什么文档」。SR 第一次问：「支撑这些输出文档的生成和管理，系统内部需要提供什么功能、满足什么约束？」

SR-F 的核心架构决策：

决策	内容	为什么架构决策
功能分类按什么组织	系统职责视角——功能按系统内部职责聚类，而非按业务部门聚类	决定了开发者看到的系统功能组织结构
末级节点指代什么	一个系统功能点——可独立分配、独立验证的系统能力单元。 在 EOS 91 规范中，biz 路径具体化为功能表单/窗口标签页（兼具配置业务系统 PA 身份），eng 路径具体化为配置页面（页面功能架构 + 操作活动）	决定了 PA 层构件划分的输入颗粒度
BA→SR 的映射方式	桥接重组（视角转换 + 分类重组），不是 1:1 分配	输出文档和 SR 的功能点可以是同一个东西的两种视角描述
功能归并规则	按系统能力域归并——同一个输出文档衍生的多个系统功能归入同一能力域节点；不同能力域的功能不共享节点	决定了系统功能的凝聚度和内聚性

视角转换示例：

BA 产出：「采购订单」输出文档——业务人员定义其字段组成、审批规则、生命周期流转
SR 产出：「订单管理功能」——支持表单录入、数据校验、库存预占、审批流驱动
（以系统职责和能力为中心，描述系统要提供什么功能来支撑这份文档的全生命周期）

这两个可以是同一个东西的两种视角：BA 的描述是给业务人员看的（「我这一步产出的是一张采购订单，里面有这些字段、这些规则」），SR 的描述是给系统设计者和开发者看的（「系统需要提供订单管理的这些能力」）。

SR-NFR 的核心架构决策：

决策	内容	为什么架构决策
STR-NFR → SR-NFR 的归并规则	同一二级分类下、不同角色来源的关注点合并为一个 SR-NFR 节点	从「按角色」切换到「按分类」——相关方验证视图 → 工程师实施视图
指标冲突的裁决规则	同一维度多个角色提了不同指标值 → 取最严格者	满足最严格的自动满足所有更宽松的

决策	内容	为什么架构决策
分类视图切换	按角色组织 → 按一级类目组织、 按二级分类归并	NFR 路径的关键视角转换——保证相关方可验证的同时工程师可查全量约束

这一层的独特特征：SR 层是设计链中第一次出现「系统内部」概念。STR 和 BA 都在描述系统外面有什么、业务上产出什么文档；SR 第一次描述系统内部要提供什么能力来支撑这些文档的全生命周期。

不可逆性：中。修改成本在文档层面，但 SR 的功能分类结构直接决定 PA 的构件划分起点。

7.6 PA 层——定义「构件的边界和契约」

这一层在解决什么设计问题：将系统功能分配和归并到可实现的产品构件中，定义构件之间的接口契约。PA 层不做新的功能设计——所有功能在 SR 层已经定义清楚。PA 层做的是**分配和归并**：100 个功能点，归并到 12 个构件里。决定功能点 A 和功能点 B 放在同一个构件还是不同构件——这就是 PA 层的架构决策。

这个决定有三个维度的约束：

- **技术合理性：**A 和 B 共享数据吗？操作同一个数据实体 → 倾向合并
- **组织可行性：**A 和 B 可以由同一个团队维护吗？需要不同技能 → 倾向拆分
- **演化独立性：**A 和 B 会以同样的节奏变更吗？A 频繁变、B 稳定 → 倾向拆分

PA 层的形式化架构决策：

决策	内容	为什么架构决策
构件怎么划分	三独立原则——可独立分配、独立替换、独立验证	构件边界是整个设计链中最不可逆的架构决策
构件边界划在哪	信息处理模式硬边界 + 业务域边界 + 数据所有权边界三者取交集	边界划错了 → 数据迁移 + 接口重定义 + 团队重组
构件之间怎么交互	接口契约——同步/异步、数据格式、超时/重试/降级策略	接口契约是构件之间的法律——一旦对外发布，修改就是破坏性变更
构件从哪来	资产优先——复用 → 改进（扩展/拆分/合并） → 新增	决定了架构决策的知识积累方式
NFR 怎么作用到构件	NFR 不分配到特定构件，作为 PA 整体约束	质量视图独立于功能分配，保证质量要求不被功能分解碎片化

PA 层最关键的一个决策——构件边界。为什么它是最不可逆的？

改了 STR 的分类维度 → 下游四层重做。但 STR 本身不改代码，改的是文档。
改了输出文档的颗粒度 → SR 重新桥接。改的是文档。
改了 SR 的功能分类 → PA 重新映射。改的是文档。

团队要重新划分 → 人员调动、知识交接、沟通链路重建。

7.7 四层决策全景对比

	STR	BA	SR	PA
解决什么问题	需求怎么组织？	业务产出什么文档？	系统功能和约束？	谁来干？
核心操作	分类 + 完备性保证	分解为输出文档	视角转换 + 归并	归并 + 分配
末级节点	端到端业务闭环	业务输出文档	系统功能点	产品构件
新增了什么？	组织维度 + 完备性判据	输出文档颗粒度判断	系统内部能力定义	构件边界 +
修改成本	文档级	文档级	文档级	代码/数据/组织
核心相关方	需求决策者	业务相关方	系统设计者	开发实施者
决策失误后果	整条链偏离	输出文档不完整	功能混乱/NFR遗漏	系统无法稳定
不可逆性	极高	中-高	中	极高

STR ← PA

↑ ↑

不可逆性极高 不可逆性极高

(整条链从这里出发) (这是最终落地处)

链的两端承受最高风险——STR 层的决策错了，后面四层全错；PA 层的决策错了，前面三层的设计在落地时需要返工。**中间两层（BA、SR）的决策修改成本在文档层面，它们的作用正是缓冲和吸收不确定性——在文档层面把问题充分暴露，避免带到代码层面。**

附录 A: 本文档与通用系统设计规范的关系

	架构基础概念与方法	通用系统设计规范
定位	概念入门和原理阐述	方法论操作规范和规则
内容	系统/架构/架构决策/架构视图的定义和原理	五层设计链的操作步骤、文档结构、验收标准

	架构基础概念与方法	通用系统设计规范
读者	任何需要建立架构认知的人	使用本方法论的从业者
依赖	不依赖任何方法论	其基本概念 (§1.1) 可引用本文档作为扩展阅读

本文档中涉及五层链的讨论（第七章）以通用系统设计规范中的定义为准，本文档仅为说明原理而引用，不代表对五层链定义的修改。

附录 B：术语速查

术语	英文	一句话定义
系统	System	一组相互关联的元素为达成共同目的而构成的整体（综合 ISO 9000 / ISO 15288 / 一般系统论）
架构	Architecture	系统的基本概念或属性，体现于元素、关系及原则（ISO 42010）
架构描述	Architecture Description	表达架构的文档/制品集合
架构视角	Architecture Viewpoint	一个观察角度——规定了关注点、相关方和模型类型
架构视图	Architecture View	从一个特定视角对架构的描述
架构决策	Architecture Decision	选错了后来要花很大代价才能改对的选择
架构风格/模式	Architecture Style/Pattern	架构决策的套路（如微服务、事件驱动、分层），不是架构本身
不可逆性	Irreversibility	修改成本高到理性决策者会选择忍受不完美的程度（经济属性）
相关方	Stakeholder	与系统有利益关系的个人、团队或组织
关注点	Concern	相关方对系统关心的某个方面
涌现属性	Emergent Property	来自元素间关系而非单个元素的系统属性（如可靠性、可扩展性）
康威定律	Conway's Law	系统结构是组织沟通结构的副本